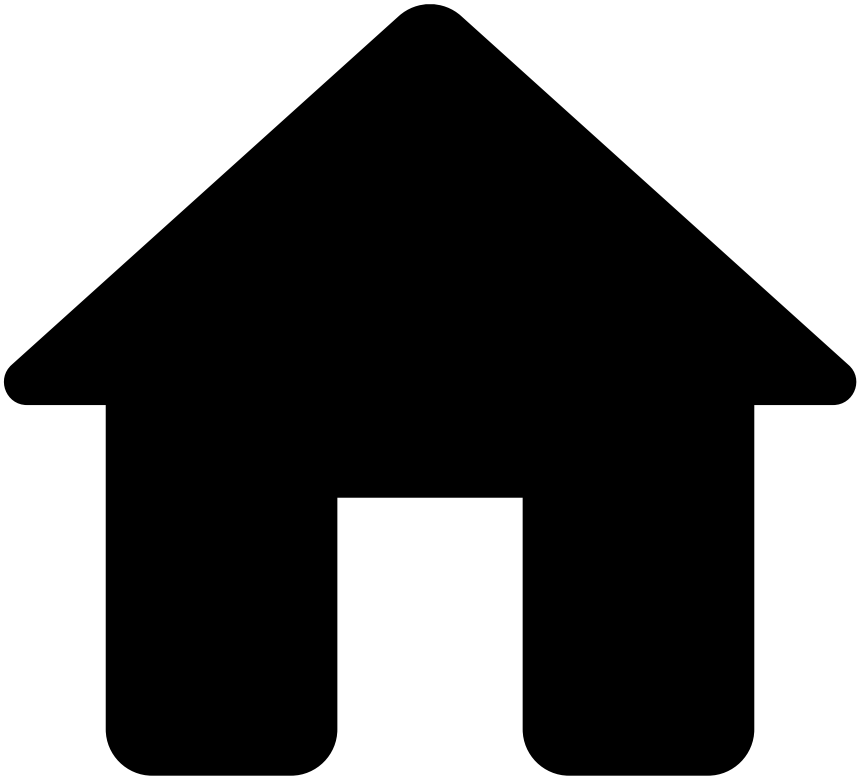


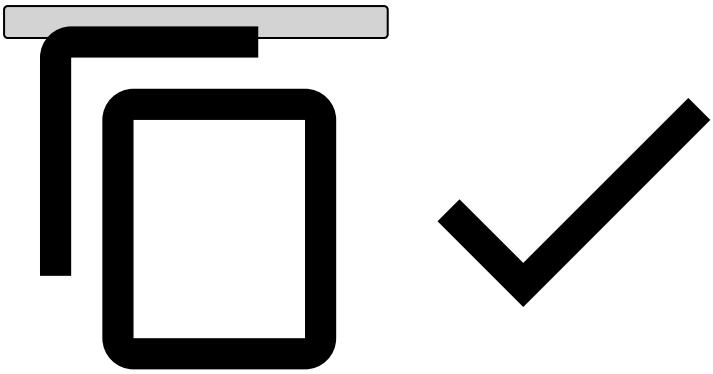
Configuring Encryption for External Components

This page walks you through the configuration of component encryption, with focus on external components.

RMI configurationâ

The RMI encryption configuration is inherited from the default one (`pulse.global.encryption.*`) unless it is overridden in `pulse.manager.rmi.encryption.*`. For example, to disable encryption on RMI communications, set this property as follows:

```
pulse.manager.rmi.encryption.trustLevel=ENCRYPTION_NONE
```



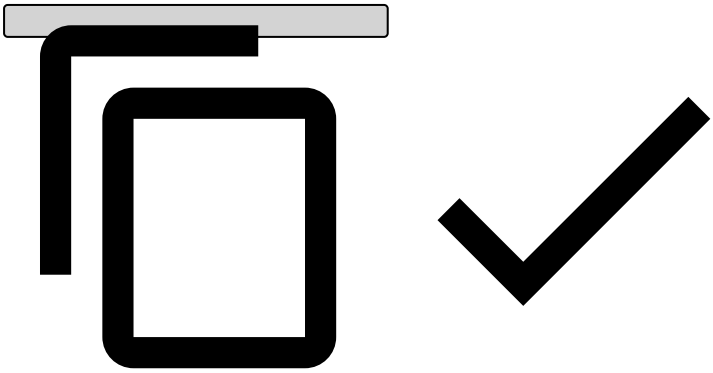
External RMI clients (e.g., custom services) need to be updated if they communicate with a Pulse server with encrypted RMI â□□see the API documentation for further information.

Pulse Management Tool configurationâ□□

The Pulse Management Tool (pulsemgmt.sh) communicates with a running Pulse server using JMX through an RMI connector. If RMI is configured to use encryption, this tool must be invoked with a correspondent client-side encryption configuration.

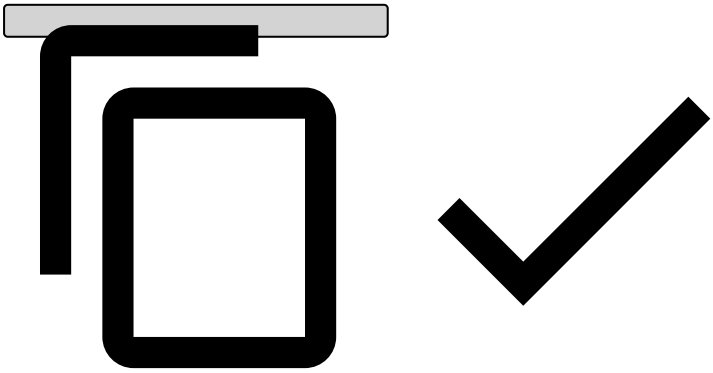
The Pulse Management Tool is completely standalone (i.e. it only manipulates MBeans), and so it doesn't access the Pulse configuration. To use it with encryption, you must set up (and system protect) a file with the encryption configuration and invoke the tool with the **-encryption** command line option, e.g.:

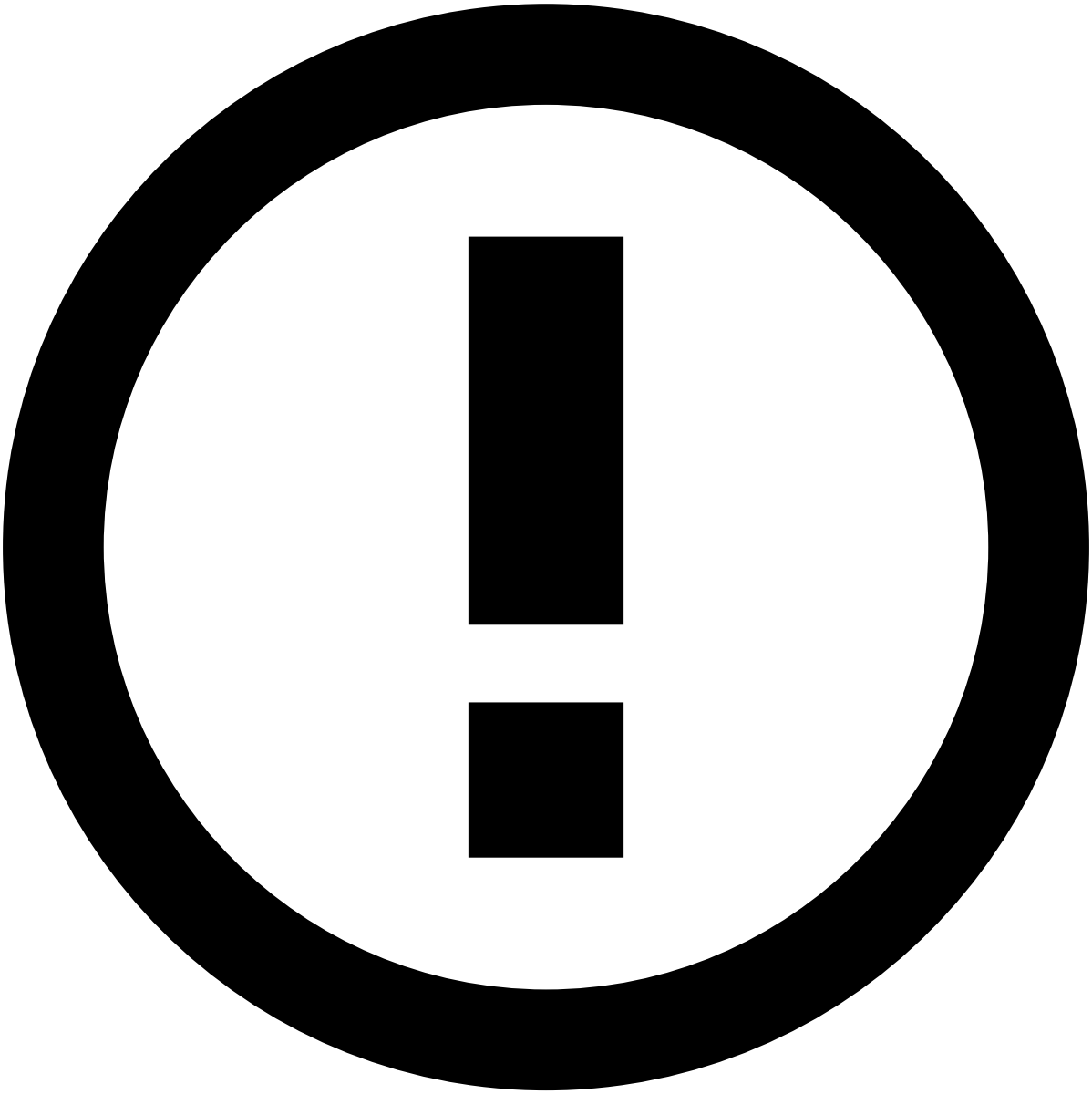
```
$ pulsemgmt.sh -shutdowncluster -encryption encrypt.properties
```



The following is an example of an encrypt.properties file:

```
 encryption.trustLevel=SERVER_AUTH
encryption.client.truststore.file=/opt/pulse/conf/engine/secret/trusted-ca.jks
encryption.client.truststore.storepwd=trusted-ca-store-pwd
```

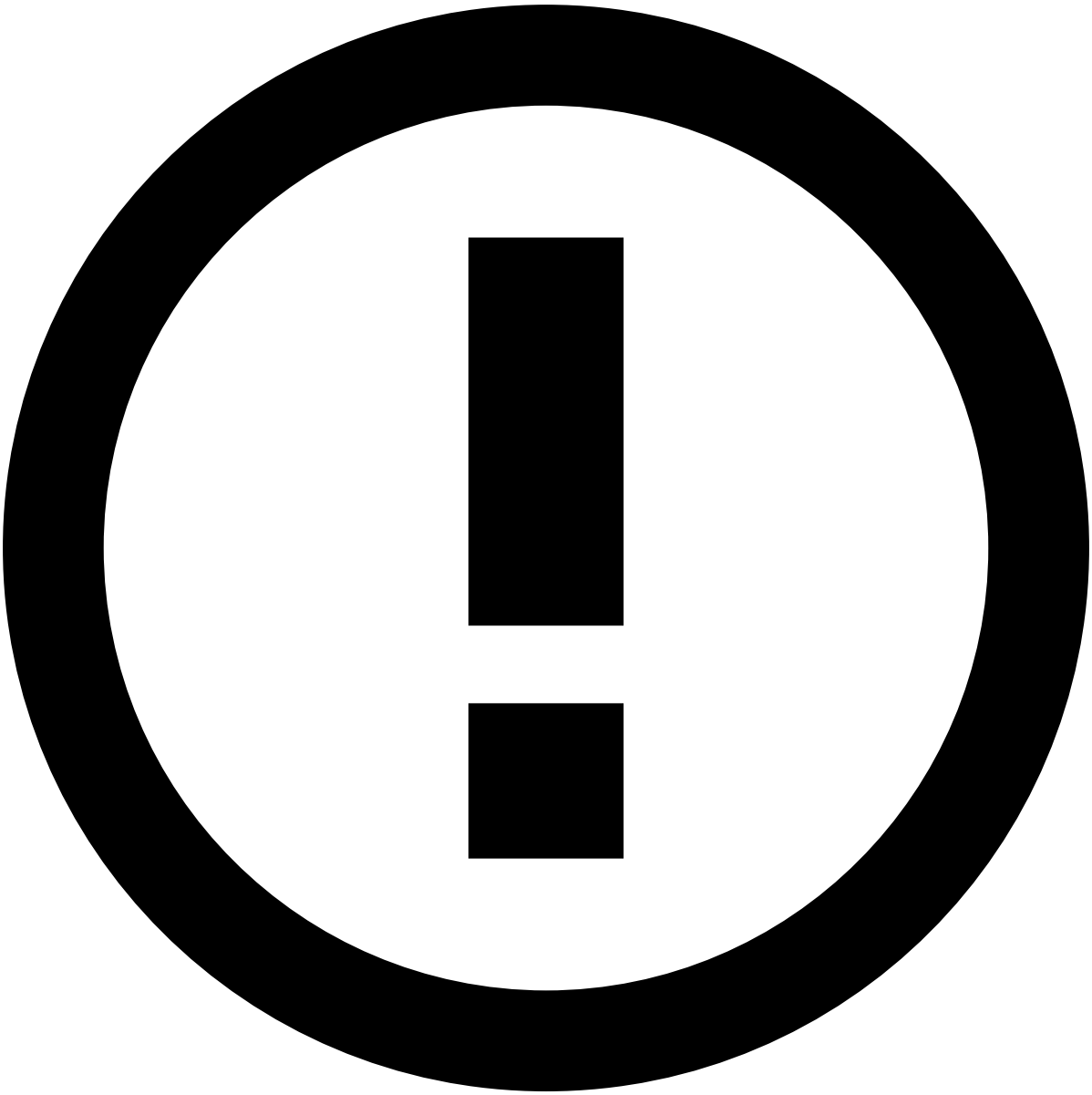




info

The `encrypt.properties` file must be protected with restricted read permissions. Note that the passwords in this file are not encrypted.

RabbitMQ configurationâ¶



Rabbit Version

Use of SSL in Rabbit was verified in version 3.6.14 / Erlang 20.1. Previous versions have known issues with SSL, see https://www.rabbitmq.com/which-erlang.md.

Rabbit requires separate configurations for each of the following channels:

- Connections between the RabbitMQ broker (server) and Pulse (client).
- Intra-datacenter broker connections, for mirroring.
- Inter-datacenter broker connections, for federation.

These are documented in the following sections.

Between broker (RabbitMQ) and Pulseâ

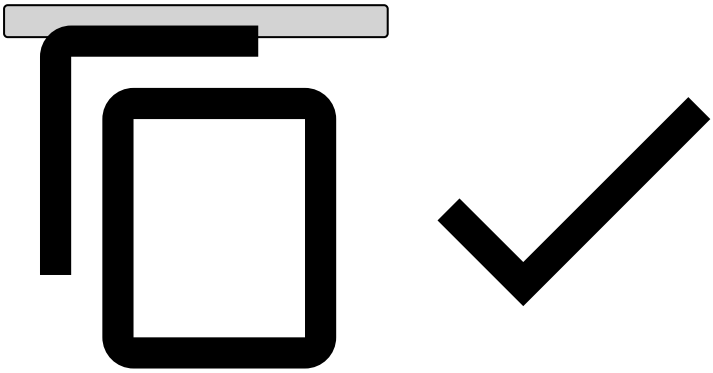
Pulse sideâ

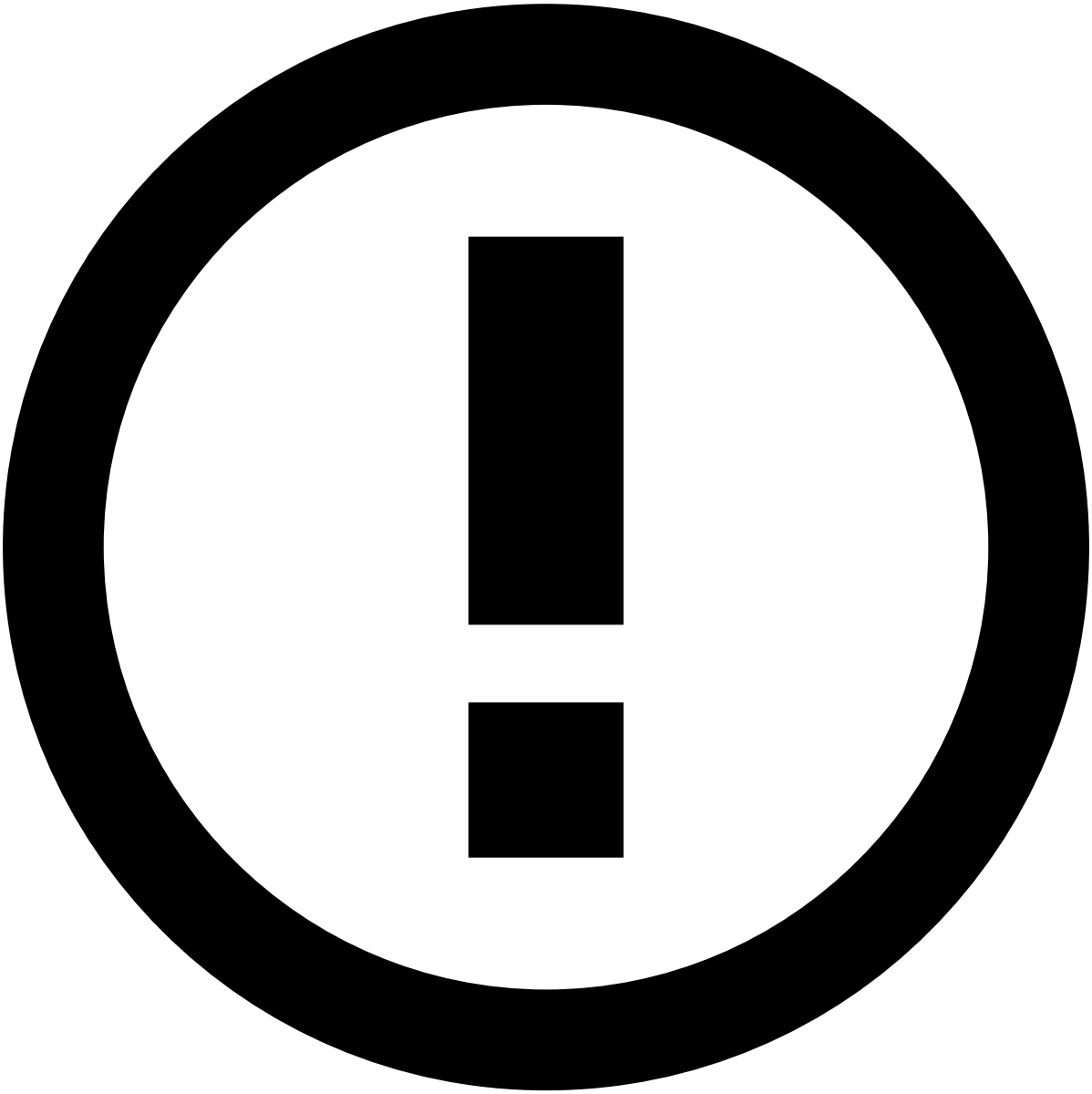
The Rabbit encryption configuration is inherited from the default client one (`pulse.global.encryption.client.*`) unless it is overridden in `pulse.manager.cluster.dc..messaging.encryption.*` . Having SSL requires the broker IPs to have the port explicitly configured in `pulse.manager.cluster.dc.messaging.brokers` . This port is defined on the Rabbit-side configuration defined below.

Rabbit sideâ

Rabbit requires the CA certificate, server certificate and server private key to be in separate files, in .PEM format. The certificate generation documentation shows how to generate these in a PKCS12 file (keystore.pkcs12). Having this file, the above PEM files can be extracted as follows:

```
 $ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nodes -nokeys -cacerts -
out cacert.pem
$ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nodes -nokeys -clcerts -
out cert.pem
$ openssl pkcs12 -in keystore.pkcs12 -passin file:secret/keystore.pwd -nocerts -out key.pem
```





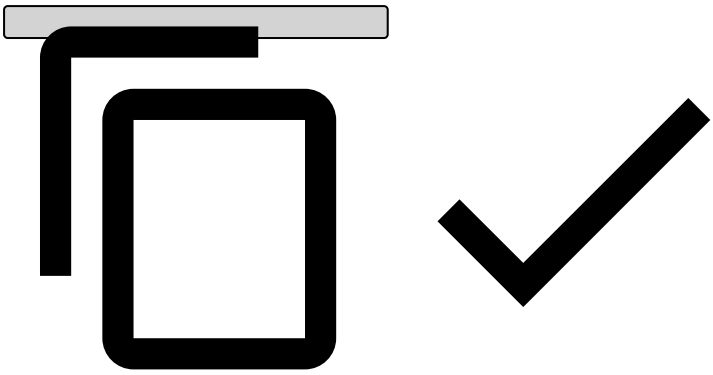
info

The key.pem password is requested interactively by openssl when introducing the third line of the above code block.

These PEM files are then configured on the rabbitmq.config file (see https://www.rabbitmq.com/ssl.html and http://erlang.org/doc/man/ssl.html for detail the `ssl_options` section):

```
[
   {rabbit, [
       {loopback_users, []},
       {ssl_listeners, [5671]},
       {ssl_options, [
           {cacertfile, "/path/to/cacert.pem"},
           {certfile, "/path/to/cert.pem"},
           {keyfile, "/path/to/key.pem"},
           {password, "<key.pem password>"},
           {verify, verify_none},
           {fail_if_no_peer_cert, false}
       ]}
```

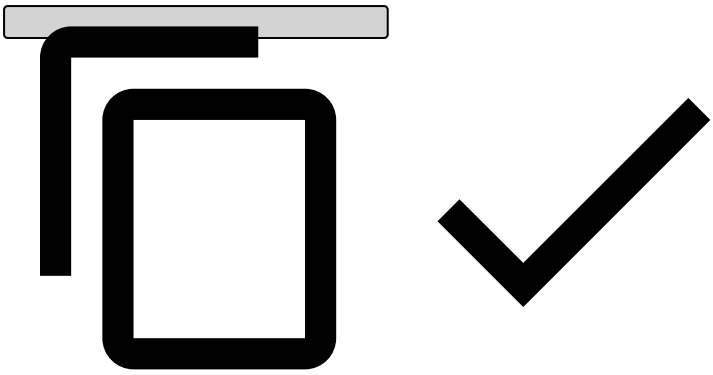
```
    ]}
].
```

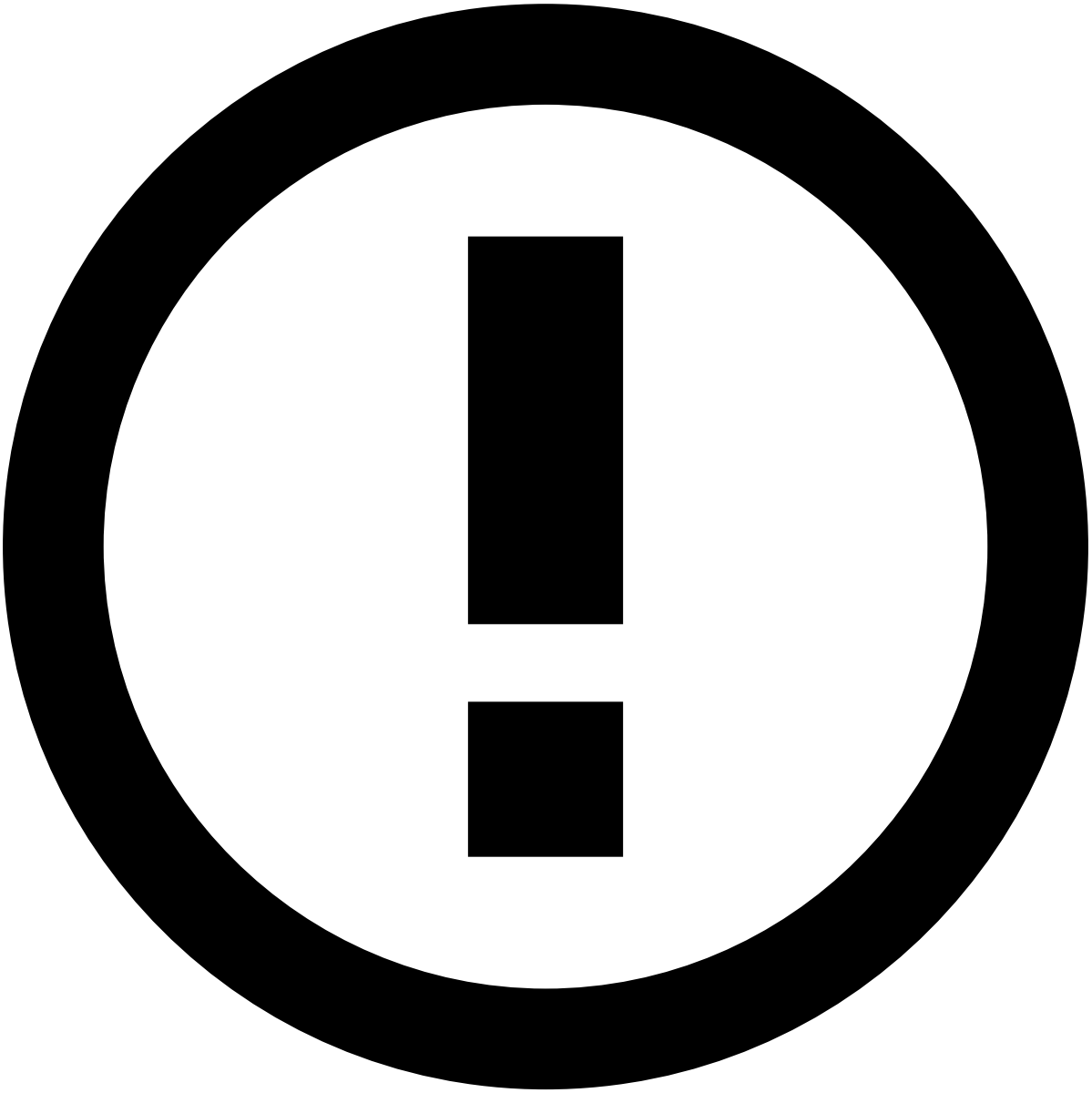


Intra-datacenter broker communicationâ□□

Intra-datacenter communication in RabbitMQ requires the certfile and unencrypted keyfile to be included in a single file:

```
 $ cat /path/to/cert.pem > /path/to/chain.pem
$ openssl rsa -in /path/to/key.pem -passin pass:<key.pem password> >> /path/to/chain.pem
```





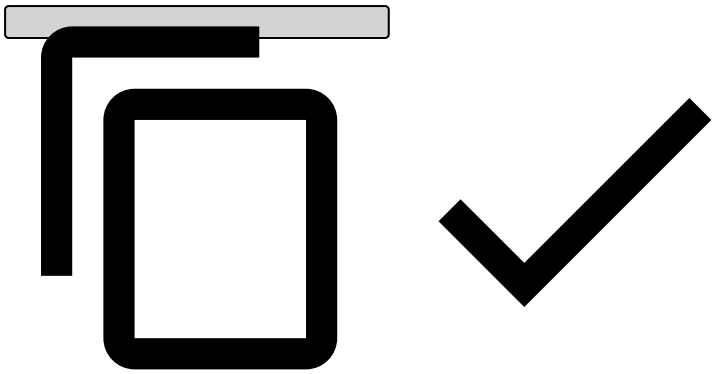
info

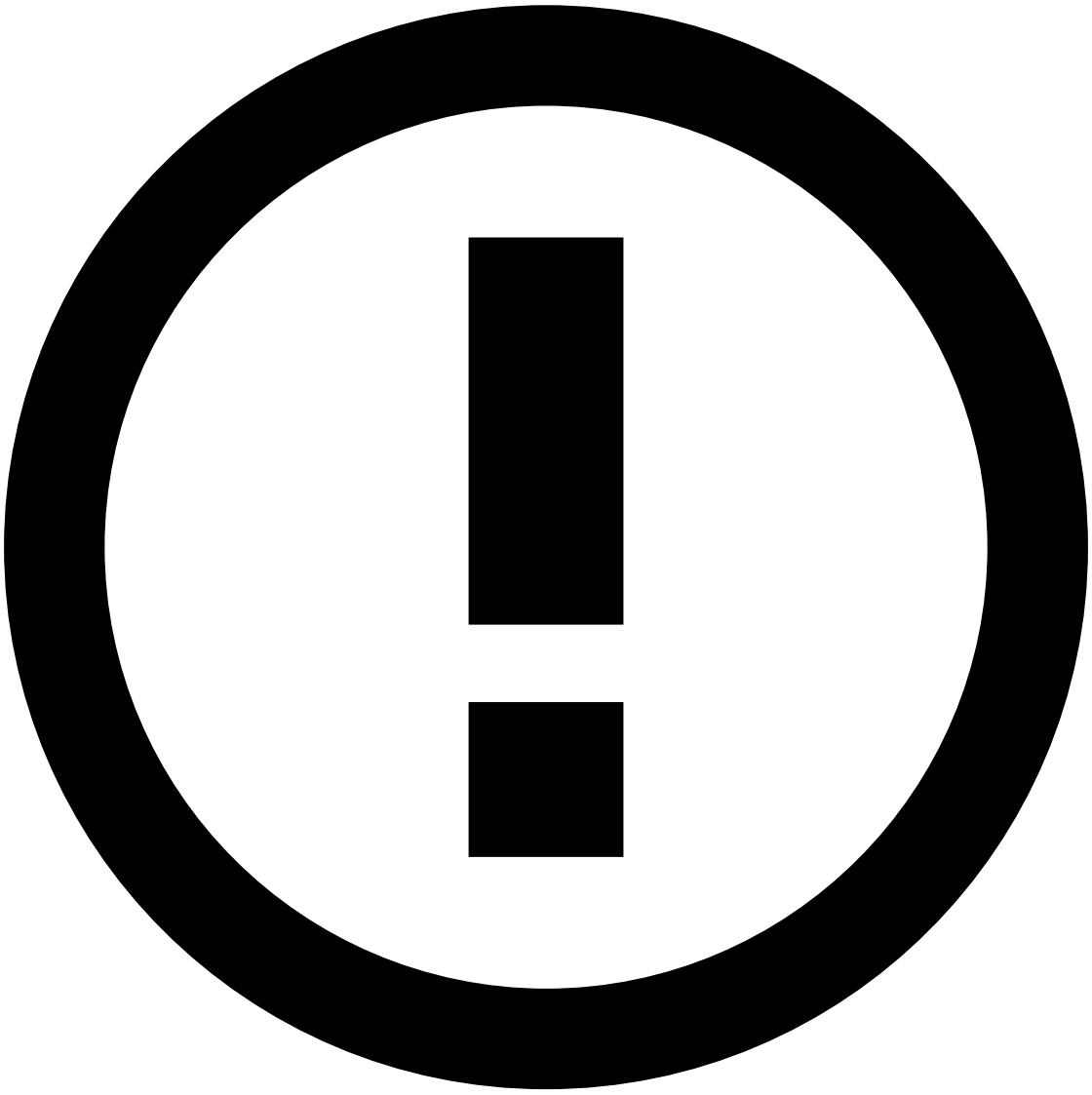
The chain.pem contains the unencrypted key, and the reading access should be restricted.

Since RabbitMQ relies on an underlying implementation of Erlang to establish its connections, some Erlang-level configurations must be provided. These are done via environment variables which are set through the rabbitmq-env.conf, which is automatically loaded when Rabbit MQ server is started.

1. The following is an implementation of using Erlang:

```
 $ echo `erl -eval 'io:format("~p", [code:lib_dir(ssl, ebin)]),halt().' -noshell`
"/path/to/erlang/<ssl-version>/ebin"
```





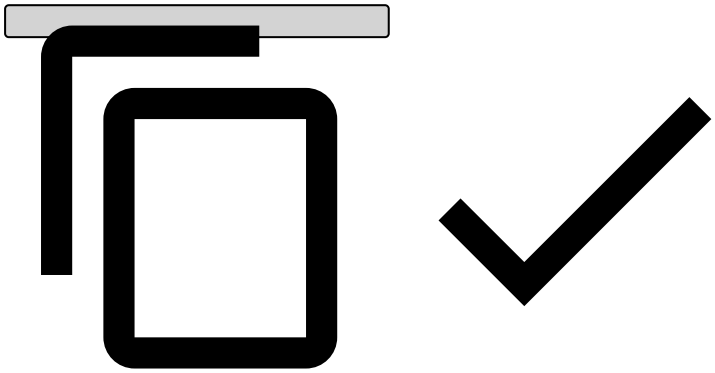
info

The output of the command above includes quotes, so it cannot be directly used via a $( ) subshell, or they would be escaped and considered part of the string.

2. Create the file `rabbitmq-env.conf` in the directory `/etc/rabbitmq`.

3. Add environment variables to the file created in step 2. Remember to replace the SSL path that you obtained in step 1. A recommended approach is to take value of the output of step 2 and set the environment variable for the Erlang path without the quotes. See an example in the next code block:

```
 ERL_SSL_PATH="/path/to/erlang/<ssl-version>/ebin"
RABBITMQ_SERVER_ADDITIONAL_ERL_ARGS="-pa ${ERL_SSL_PATH} -proto_dist inet_tls -
ssl_dist_opt server_certfile /path/to/chain.pem -ssl_dist_opt server_secure_renegotiate
true client_secure_renegotiate true"
RABBITMQ_CTL_ERL_ARGS="-pa ${ERL_SSL_PATH} -proto_dist inet_tls -ssl_dist_opt
server_certfile /path/to/chain.pem -ssl_dist_opt server_secure_renegotiate true
client_secure_renegotiate true"
```



See https://www.rabbitmq.com/clustering-ssl.html for RabbitMQ's own information.

Inter-datacenter broker communication (federation)â□□

Configuring federation with encryption requires the following changes relative to a non-encrypted setup:

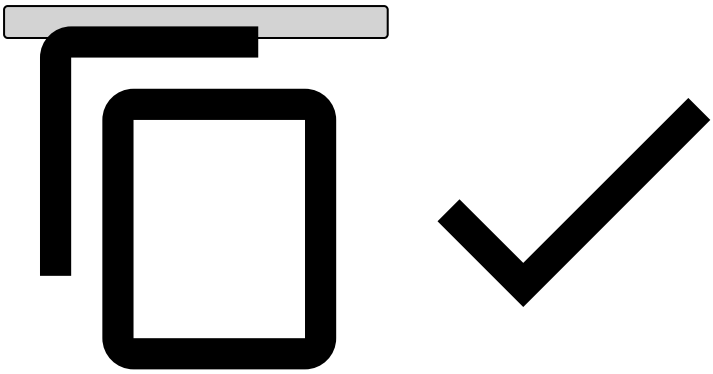
1. The protocol changes from "amqp" to "amqps"
2. The address now receives the explicit port (:5671)

An example of SSL federation URI with the changes already applied is provided below:

```
 # RabbitMQ Cluster 1
rabbitmqctl set_parameter federation-upstream my-upstream \
    '{"uri":["amqps://<user>:<pass>@<server in cluster 2>:5671", "amqps://<user>:<pass>@<server
in cluster 2>:5671", ...], "ack-mode":"no-ack"}'
rabbitmqctl set_policy --apply-to exchanges federate-me "^global-"\
    '{"federation-upstream":"my-upstream"}'

# RabbitMQ Cluster 2
rabbitmqctl set_parameter federation-upstream my-upstream \
    '{"uri":["amqps://<user>:<pass>@<server in cluster 1>:5671", "amqps://<user>:<pass>@<server
in cluster 1>:5671", ...], "ack-mode":"no-ack"}'
rabbitmqctl set_policy --apply-to exchanges federate-me "^global-" \
    '{"federation-upstream":"my-upstream"}'

# Make federation queues replicated so they are also highly available
rabbitmqctl set_policy replicate-queues-federation "^federation" '{"ha-mode":"all"}' --apply-to
queues
```



Kafkaâ

Kafka supports all encryption trust levels defined in Pulse. As a Java-based system, it supports the standard Java certificate formats used in Pulse and other Java-based components.

Pulse sideâ

The Kafka encryption configuration is inherited from the default client configuration defined by properties with the `pulse.global.encryption.client.*` prefix. However, it can be overridden by configurations defined with the `pulse.manager.cluster.dc.<dc>.messaging.encryption.*` prefix. In addition to the standard Pulse encryption, Kafka also supports server authentication based on the IP address of the certificate. This is not enabled by default. To enable it, set the `ssl.endpoint.identification.algorithm` raw property to *https*.

Broker sideâ

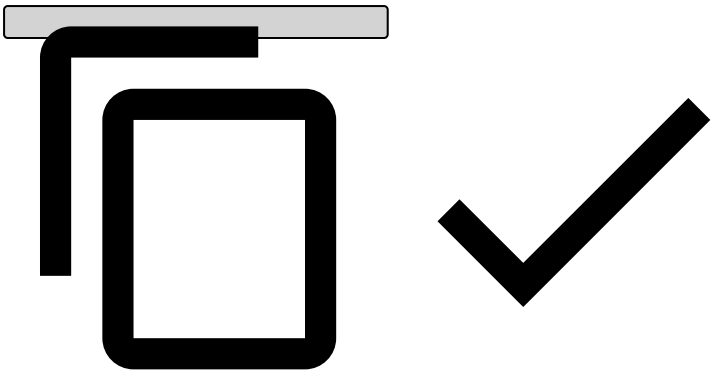
The Kafka broker configuration is very similar to the Pulse/Server configuration. Server authentication requires a keystore in jks format, whereas client authentication requires a truststore for the trusted clients. These are configured in *server.properties* as shown below (for server authentication only):

```
 # Listener protocol must be either SSL or SASL_SSL, depending on SASL authentication being used or not
listeners=SASL_SSL://172.17.0.3:9092

# Key and truststore properties
ssl.keystore.location = /path/to/keystore.jks
ssl.keystore.password = <the.keystore.password>
ssl.key.password = <the.keystore.private.key.password>
ssl.truststore.location = /path/to/trustedca.jks
ssl.truststore.password = <the.truststore.password>

# Disable hostname verification
ssl.endpoint.identification.algorithm=

# Needed because otherwise kerberos is assumed
sasl.enabled.mechanisms=PLAIN
sasl.mechanism.inter.broker.protocol=PLAIN
```



This configuration includes SSL on inter-broker communications.

Mirror-makerâ□□

Mirror-maker is used on multi-datacenter setups to achieve topic federation between data centers. Should SSL be used, then the *server.properties* SSL and SASL properties described earlier in this page must also be specified in the mirror-maker *consumer.properties* and *producer.properties* files.

Relational DB configurationâ□□

Postgresâ□□

Server configurationâ□□

Enabling SSL on a postgres server requires a server certificate and private key with unprotected PEM format. This can be extracted using the same procedure as the one described above for Rabbit.

These files can be installed on the server as described in https://www.postgresql.org/docs/9.5/static/ssl-tcp.html.

Client (Pulse) configurationâ□□

The client side is configured at the JDBC connection string level with two available options (see https://jdbc.postgresql.org/documentation/91/ssl-client.html):

- **Without server certificate validations:** Connection URL parameter `ssl=true&sslfactory=org.postgresql.ssl.NonValidatingFactory` needs to be defined. This skips server certificate validations, and thus, does not require a truststore. However, this option is less secure.
- **With server certificate validations:** Connection URL parameter ssl=true needs to be defined as well as the two system properties `javax.net.ssl.trustStore` and `javax.net.ssl.trustStorePassword`. The last two system properties only need to be defined if the global pulse configuration is not set.

Oracleâ□□

Client (Pulse) configurationâ□□

The Oracle JDBC thin driver supports JSSE encryption through its standard `javax.net.ssl.*` properties, which are automatically set on all Pulse components as long as a global encryption configuration is defined. The Pulse side configuration requires the following:

- JKS keystore and truststore configured in the `pulse.global.encryption.*` properties.
- *tnsnames* definition in the connection URL specifying the tcps protocol, as described in https://docs.oracle.com/cd/B19306_01/java.102/b14355/sslthin.htm#BABHFBJD.

Both server and client authentication are supported.

Cassandra configurationâ□□

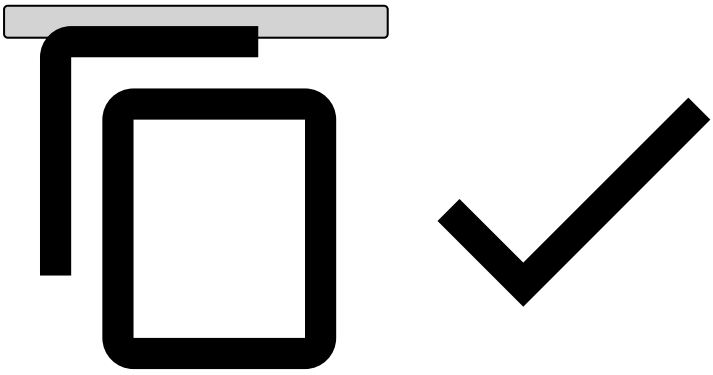
The full Datastax documentation can be found here.

Client (Pulse) to nodeâ□□

Cassandra configuration in cassandra.yaml is very similar to the Pulse/Server configuration. Server authentication requires a keystore in jks format whereas client authentication requires a truststore for the trusted clients. These are configured in the cassandra.yaml client_encryption_options as shown below (for server authentication only):

```
 # enable or disable client/server encryption.
client_encryption_options:
    enabled: true
    # If enabled and optional is set to true encrypted and unencrypted connections are handled.
    optional: false
    keystore: path/to/keystore.jks
    keystore_password: <the.keystore.password>
    # require_client_auth: false
    # Set trustore and truststore_password if require_client_auth is true
    # truststore: conf/.truststore
    # truststore_password: cassandra

    # More advanced defaults below:
    # protocol: TLS
    # algorithm: SunX509
    # store_type: JKS
    # cipher_suites:
[TLS_RSA_WITH_AES_128_CBC_SHA,TLS_RSA_WITH_AES_256_CBC_SHA,TLS_DHE_RSA_WITH_AES_128_CBC_SHA,TLS_
DHE_RSA_WITH_AES_256_CBC_SHA,TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA,TLS_ECDHE_RSA_WITH_AES_256_CBC_S
HA]
```



The keystore file can be the same as in Pulse as long as it is signed with the CA that Pulse trusts. Additional certificates are not required on the Pulse side.

The encryption configuration on the Pulse side can be specified either under `pulse.global.encryption.client.*` (or under `pulse.global.cassandra.encryption.*` if a specific configuration is required for Cassandra).

Node-to-nodeâ□□

Node-to-node encryption is configured similarly to the client-to-node encryption. Client-to-node is defined in the client_encryption_option, whereas the node-to-node is defined in the server_encryption_options section.

As with the node-to-node configuration, JKS key and trust stores may be required.

See https://docs.datastax.com/en/cassandra/3.0/cassandra/configuration/secureSSLNodeToNode.html for details.

Jetty server / REST API configurationâ□□

REST API encryption can be configured with self-signed certificates (without CA) as described here. This method can be used in a scenario in which only Jetty SSL is configured. However, the certificates generated by the scripts described in the page Creating Certificates could be used instead. The scripts are configured in `$PULSE_HOME/conf/engine/jetty/jetty.xml` as defined here.

Spark configurationâ

Configuring communications security on Spark requires addressing:

- Communications between Pulse code running on Spark Executors and Pulse.
- Communications between Pulse code running on Spark Executors and the Spark Cluster (e.g., HDFS access on data source analysis).
- Communications between Pulse and the spark cluster (e.g., launching jobs, HDFS access on datas source previews).
- Communications between cluster components. These depend on the containers where spark is deployed.

These are documented in the following sections.

Securing communications between Pulse executors and Pulseâ

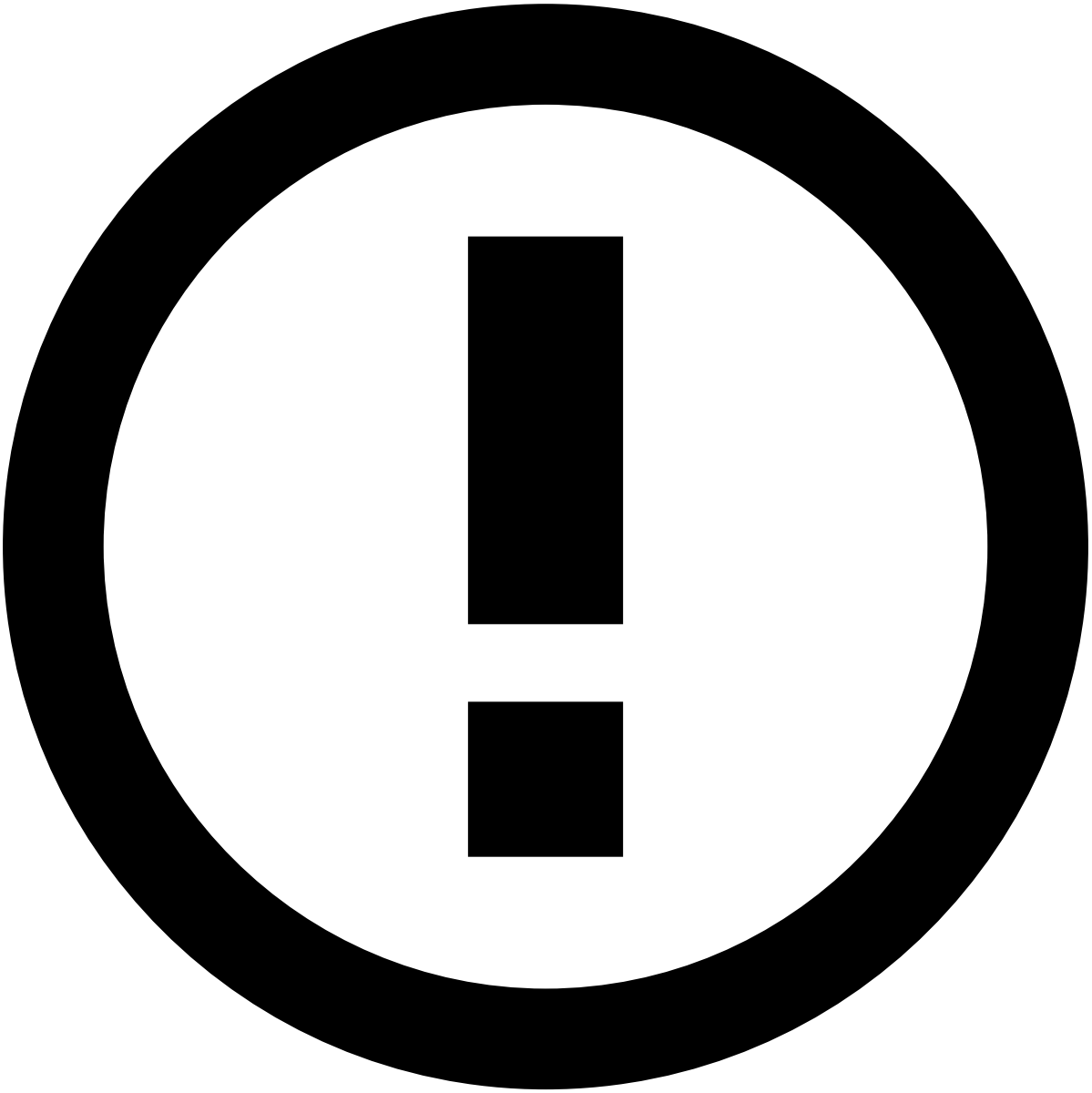
Pulse jobs submitted to Spark need to communicate with Pulse through RMI or access event/profile storage, and so they require truststores (and possibly keystores if client authentication is used). By default, executors inherit the encryption configuration defined in pulse.global and the event/profile storage config, and thus a truststore must be present on the configured location and with the same passwords. This truststore can be the same as the one specified for Spark internal protocols.

Spark standaloneâ

Protocols over SSLâ

Pulse sideâ

SSL security configuration passed to Spark when submitting jobs is specified, in Pulse, under the `pulse.global.services.modules.datascience.spark.forward.spark.ssl.*` space.



info

The `spark.*` properties mentioned in this section are set under `pulse.global.services.modules.datascience.spark.forward.*`.

The properties below inherit the settings set in the Pulse global encryption configuration if not set explicitly:

- spark.ssl.protocol
- spark.ssl.keyPassword
- spark.ssl.keyStore
- spark.ssl.keyStorePassword
- spark.ssl.trustStore
- spark.ssl.trustStorePassword

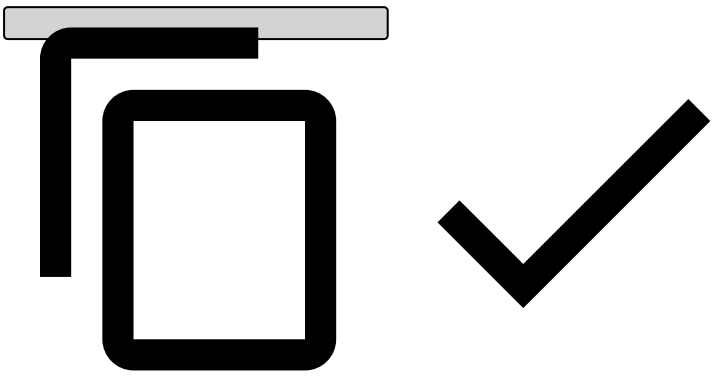
The SSL properties above specify default properties for the following protocols (Spark 1.6.3):

- fs (for file distribution).
- akka (essentially job control).

See https://spark.apache.org/docs/1.6.3/security.html#ssl-configuration for additional encryption properties and instructions on how to configure each protocol separately.

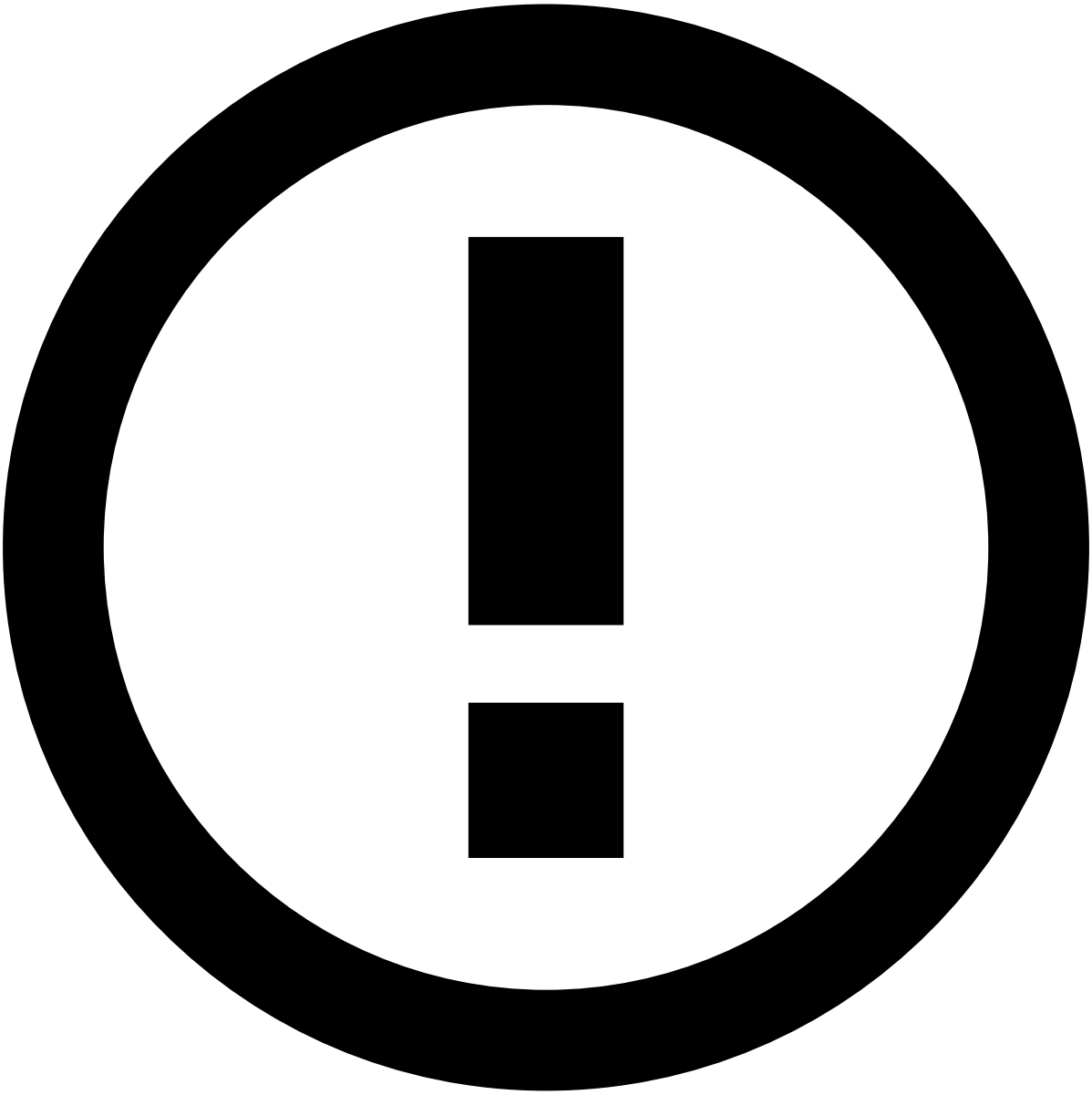
Since the properties above are automatically set by Pulse, to enable SSL the only setting to be configured is the following:

```
spark.ssl.enabled=true
```



Note that Spark 1.6.3 does not support at-rest encryption of temporary files such as shuffle files and cached data.

Spark sideâ□□

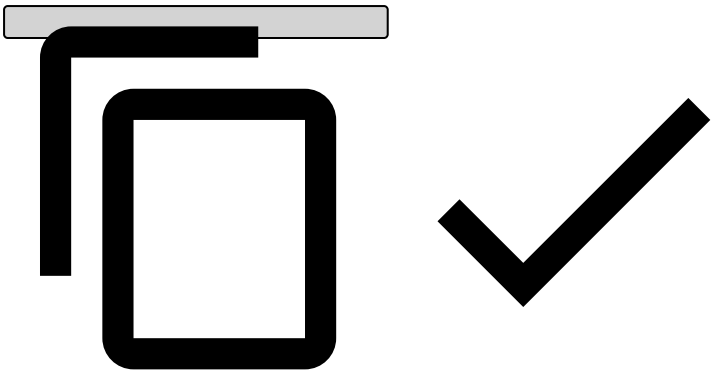


info

The Spark nodes require `openssl` to be installed in order to be able to create the connection between them, even before Pulse is started.

The following needs to be configured on each Spark node (conf/spark-defaults.conf file) for SSL to be used between nodes:

```
 spark.ssl.enabled true
spark.ssl.keyPassword <Password>
spark.ssl.keyStore /path/to/keystore.jks
spark.ssl.keyStorePassword <KeyStorePassword>
spark.ssl.protocol TLS
spark.ssl.trustStore /path/to/trustedca.jks
spark.ssl.trustStorePassword <TrustStorePassword>
```



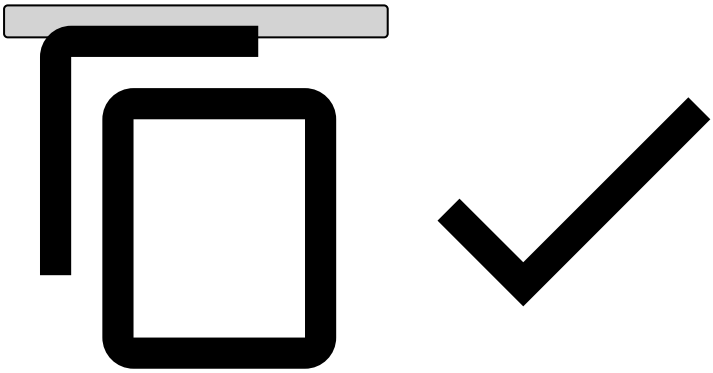
Protocols over TCP with SASLâ□□

The RCP protocol is implemented over SASL and must be encrypted because store sensitive properties are passed over it. On a setup with no user authentication (i.e., kerberos), a shared secret string must be configured in Pulse and Spark nodes.

Pulse sideâ□□

SASL is enabled with the following Pulse properties:

```
 pulse.global.services.modules.datascience.spark.forward.spark.authenticate=true
pulse.global.services.modules.datascience.spark.forward.spark.authenticate.secret=<SECRET>
pulse.global.services.modules.datascience.spark.forward.spark.authenticate.enableSaslEncryption=t
rue
pulse.global.services.modules.datascience.spark.forward.spark.network.sasl.serverAlwaysEncrypt=t
rue
```



Spark sideâ□□

The following must also be configured in each Spark container machine, by creating/modifying the conf/spark-defaults.conf file in the Spark nodes:

```
 spark.authenticate true
spark.authenticate.secret <SECRET>
spark.authenticate.enableSaslEncryption true
spark.network.sasl.serverAlwaysEncrypt true
```